

JavaScript Essentials for PHP Developers

— Study Notes

Course: Laracasts — JavaScript Essentials for PHP Developers (35 episodes) **Audience:** PHP developers learning JavaScript **Focus:** Where JS overlaps PHP, where it diverges, and the idioms that actually matter day-to-day.

How to use these notes

Each episode has the same five sections:

- **TL;DR** — what the episode is about
 - **PHP → JS differences** — direct side-by-side where useful
 - **Concepts, explained** — runnable code, naive versions where they teach something
 - **Gotchas & warnings** — the traps that bite PHP devs specifically
 - **Mental model** — one paragraph to cement the takeaway
 - **Try it yourself** — a small hands-on exercise
-

Episode 1 — Structure, Syntax, And Scope

TL;DR

JavaScript's syntax looks almost identical to PHP at a glance — braces, semicolons, if/else, functions — but the rules underneath are different. This episode maps the visible similarities and flags the biggest early traps: accidental globals, string quoting, variable scoping, and method-call syntax.

PHP → JS differences

- Variables have **no** `$` **sigil** in JS — just `name`, not `$name`
- Strings: **no interpolation difference** between `"..."` and `'...'` (unlike PHP where `"..."` expands `$vars`)
- Method calls use `.` not `->`

- String concatenation uses `+` not `.`
- The naming convention is **camelCase**, not `snake_case`

Concepts, explained

The browser's `alert()`

The simplest way to see output from JS running in a browser. It pops a modal dialog.

```
alert('Hello, world!');
```

Function declaration

Like PHP, but no `$` and camelCase by convention.

```
function helloWorld() {  
    return 'Hello, world!';  
}  
  
alert(helloWorld());
```

PHP analogue:

```
function helloWorld() {  
    return 'Hello, world!';  
}
```

Strings — quotes are interchangeable

Unlike PHP, there is no behavioural difference between `'...'` and `"..."`.

```
const a = 'single';  
const b = "double";  
const name = 'Ben';  
  
// Neither interpolates. Both are literal text.  
console.log('Hello, $name'); // Hello, $name  
console.log("Hello, $name"); // Hello, $name
```

Semicolons are optional but recommended

JS has "automatic semicolon insertion" but relying on it leads to subtle bugs. Always write them.

```
const x = 1;    // recommended  
const y = 2    // works, but don't
```

Conditionals and throwing errors

```
function divide(a, b) {  
    if (b === 0) {  
        throw new Error('Cannot divide by zero');  
    }  
    return a / b;  
}
```

Accidental globals — the big footgun

If you assign to a variable **without** `var`, `let`, or `const`, JS silently creates a global. This is the single most common early bug for PHP devs.

```
function greet() {  
  message = 'hello'; // WRONG — no declaration, becomes a global  
}  
  
greet();  
console.log(message); // 'hello' — leaked out of the function
```

Correct version:

```
function greet() {  
  const message = 'hello'; // scoped to the function  
  return message;  
}
```

`var` — function-scoped, closest to PHP

```
function demo() {  
  var x = 1;  
  if (true) {  
    var x = 2; // SAME x, not a new one — var is function-scoped  
  }  
  console.log(x); // 2  
}
```

Everything is an object — method syntax

Strings, numbers, arrays all have methods accessed with `.`

```
const name = ' Benjamin ';  
console.log(name.trim()); // 'Benjamin'  
console.log(name.toUpperCase()); // ' BENJAMIN '  
console.log(name.trim().toUpperCase()); // 'BENJAMIN'
```

Gotchas & warnings

- **Accidental globals.** Forgetting `let` / `const` / `var` silently pollutes the global scope. In "strict mode" this throws, but in plain scripts it doesn't.
- `.` **vs** `->`. Muscle memory from PHP will have you typing `$obj->method()` — it won't compile.
- **Interpolation in single/double quotes doesn't exist.** For interpolation you need backticks (covered in Ep 3).

Mental model

JS *looks* like PHP with `$` removed, but the variable-declaration keywords (`var`, `let`, `const`) carry real scoping meaning. Always declare. Never assign to an undeclared name.

Try it yourself

Write a function `greet(name)` that returns `"Hello, NAME!"` in uppercase. Call it with your own name

and alert the result.

Episode 2 — Data Types

TL;DR

JS is dynamically and weakly typed — no type hints, aggressive coercion. Seven core types to know: `undefined`, `string`, `number`, `boolean`, `symbol`, `object`, and `function`. `null` is an oddity: it's technically an object per `typeof`, which is a well-known historical bug.

PHP → JS differences

- **No separate int/float** — all numbers are one `number` type (64-bit float)
- **No type hints at all** — the language won't enforce types for you
- **Associative arrays don't exist** — use objects for key/value data
- **Functions are a type** — first-class values, not just language constructs

Concepts, explained

Primitive types

```
let a;           // undefined – declared, no value
let b = 'hello'; // string
let c = 42;       // number (integer)
let d = 3.14;     // number (float – same type!)
let e = true;     // boolean
let f = Symbol('id'); // symbol (rarely used day-to-day)
```

`undefined` vs `null`

- `undefined` = "I was declared but never given a value"
- `null` = "I was intentionally set to nothing"

```
let notSet;           // undefined (implicit)
let intentional = null; // null (explicit "nothing")

console.log(notSet);   // undefined
console.log(intentional); // null
```

Non-primitive types

```
const obj = {};           // object literal
const obj2 = new Object(); // same thing, verbose form
const arr = [1, 2, 3];    // array (really a special kind of object)
const fn = function () {}; // function (first-class value)
```

typeof — what is this?

```
typeof undefined;    // 'undefined'
typeof 'hi';          // 'string'
typeof 42;            // 'number'
typeof true;          // 'boolean'
typeof Symbol();      // 'symbol'
typeof {};            // 'object'
typeof [];            // 'object' ← trap
typeof null;          // 'object' ← trap (historical bug)
typeof function(){};  // 'function'
```

Detecting arrays properly:

```
Array.isArray([1, 2, 3]); // true
Array.isArray('hello');   // false
```

Arrays are index-based, not associative

PHP conflates these. JS does not.

```
// Good: arrays for ordered lists of values
const fruits = ['apple', 'banana', 'cherry'];

// Bad: using arrays as key/value stores (anti-pattern in JS)
const user = [];
user['name'] = 'Ben'; // works, but wrong idiom

// Good: use an object for key/value
const userCorrect = { name: 'Ben', age: 23 };
```

Gotchas & warnings

- `typeof null === 'object'`. Legendary bug; every JS dev has to remember it.
- `typeof []` is also `'object'`. Use `Array.isArray()` to test for arrays.
- **Numbers don't split into int/float.** `10 / 3` gives `3.3333...`, there's no integer division without `Math.floor()` or `Math.trunc()`.
- **Associative-array habits from PHP.** If you catch yourself writing `arr['key'] = 'val'`, stop and use an object.

Mental model

Two buckets: **primitives** (copied by value) and **non-primitives** (passed by reference). `null` is a primitive that `typeof` lies about. Numbers are one type — JS doesn't care about your int-vs-float feelings.

Try it yourself

Declare one variable of each primitive type. Log `typeof` each one. Then log `typeof null` and `typeof []` and explain why both say `'object'`.

Episode 3 — Smooth Operators

TL;DR

Most operators match PHP, but `+` is overloaded (arithmetic + concatenation), which opens the door to painful coercion bugs. Use `===` over `==`, always. Template literals with backticks replace PHP's string interpolation.

PHP → JS differences

- No spaceship operator (`<=>`)
- Concatenation is `+`, not `.`
- Template strings use backticks with `${...}`, not `"$var"`
- `==` does type coercion — always prefer `===`

Concepts, explained

`+` does double duty

```
1 + 2;           // 3           (both numbers → arithmetic)
'a' + 'b';       // 'ab'       (both strings → concatenation)
1 + '2';         // '12'       (mixed → string concatenation wins!)
1 + 2 + '3';     // '33'       (left-to-right: 1+2=3, then 3+'3'='33')
'3' + 1 + 2;     // '312'      (string starts left, all become strings)
```

Other math operators coerce strings to numbers

```
'10' - 1;       // 9
'10' * 2;       // 20
'10' / 2;       // 5
'abc' - 1;      // NaN — not a number
```

Template literals (backticks)

```
const name = 'Benjamin';
const age = 23;

// Old-school concatenation
const greeting1 = 'Hello, ' + name + '! You are ' + age + '.';

// Template literal — cleaner, supports multiline
const greeting2 = `Hello, ${name}! You are ${age}.`;

const multi = `Line one
Line two
Line three`;
```

Equality: `==` vs `===`

```
0 == '';         // true (coerces!)
0 == '0';        // true
null == undefined; // true
```

```
'5' == 5;      // true
0 === '';     // false
0 === 0;      // true
'5' === 5;    // false ← always want this behaviour
```

Rule: never use `==` in new code. Always `===`.

Truthy and falsy values

Falsy: `false`, `0`, `''`, `null`, `undefined`, `NaN`. Everything else — including `'0'`, `'false'`, `[]`, `{}` — is truthy.

```
if ('') console.log('no');    // not logged (falsy)
if ('0') console.log('yes');  // logged! ('0' is truthy — PHP difference!)
if ([]) console.log('yes');    // logged ([] is truthy in boolean context)
```

Spread and rest

Spread unpacks an iterable. Rest packs args into an array.

```
// Spread — unpack
const a = [1, 2, 3];
const b = [...a, 4, 5]; // [1, 2, 3, 4, 5]

// Rest — pack
function sum(...nums) {
  return nums.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4); // 10
```

Logical operators

```
true && false; // false
true || false; // true
!true;         // false
```

Gotchas & warnings

- `1 + '2' === '12'`. If a string touches `+`, you're concatenating.
- `'0'` is truthy in JS, falsy in PHP. Classic migration bug.
- `==` lies. `0 == ''`, `null == undefined`, and other absurdities. Use `===`.

Mental model

`+` is the only operator that can't make up its mind — it reads the types and picks. Every other arithmetic operator says "I want numbers, I'll coerce if needed." Comparisons with `===` ask "exact same type and value?" — that's almost always what you want.

Try it yourself

Predict the output of each, then run:

```
console.log(1 + 2 + '3');
```

```
console.log('1' + 2 + 3);
console.log('5' - 2);
console.log(0 == false);
console.log(0 === false);
```

Episode 4 — Controlling Flow

TL;DR

Almost all control flow constructs (`if` , `switch` , `for` , `while`) are identical to PHP. The new additions worth knowing: `for...of` for array values and `for...in` for object keys. JS objects are what PHP calls associative arrays.

PHP → JS differences

- `for...of` iterates values of an array (like PHP's `foreach ($arr as $val)`)
- `for...in` iterates keys of an object (like PHP's `foreach ($arr as $key => $val)`) but you only get the key)
- Objects are iterable via `for...in` since they're the associative-array equivalent

Concepts, explained

`if / else if / else`

```
const score = 72;
if (score >= 90) {
  console.log('A');
} else if (score >= 80) {
  console.log('B');
} else {
  console.log('C or below');
}
```

Ternary

```
const age = 20;
const type = age >= 18 ? 'adult' : 'minor';
```

`switch`

```
const day = 'mon';
switch (day) {
  case 'mon':
  case 'tue':
  case 'wed':
  case 'thu':
```



```

    case 'fri':
      console.log('weekday');
      break;
    case 'sat':
    case 'sun':
      console.log('weekend');
      break;
    default:
      console.log('unknown');
  }

```

Classic `for`

```

for (let i = 0; i < 5; i++) {
  console.log(i);
}

```

`for...of` — iterate array values

```

const fruits = ['apple', 'banana', 'cherry'];
for (const fruit of fruits) {
  console.log(fruit);
}
// apple
// banana
// cherry

```

`for...in` — iterate object keys

```

const person = { firstName: 'Ben', age: 23, city: 'Pretoria' };
for (const key in person) {
  console.log(`${key}: ${person[key]}`);
}
// firstName: Ben
// age: 23
// city: Pretoria

```

Note the **bracket access** `person[key]` — you can't use dot notation with a variable.

`while` and `do...while`

```

let i = 0;
while (i < 3) {
  console.log(i);
  i++;
}

let j = 0;
do {
  console.log(j);
  j++;
} while (j < 3);

```

`break` and `continue`

```

for (let i = 0; i < 10; i++) {
  if (i === 3) continue; // skip 3
  if (i === 7) break;    // stop at 7
  console.log(i);
}
// 0 1 2 4 5 6

```

Gotchas & warnings

- **for...in on an array is almost always a bug.** You get string keys (`'0'` , `'1'`) and inherited enumerable properties. Use `for...of` or a plain `for` loop for arrays.
- **switch requires break** (or `return`) or it falls through — identical to PHP, but easy to forget.
- **Bracket notation** is mandatory when the key is a variable: `obj[key]` , not `obj.key` .

Mental model

Arrays have values you iterate over — `for...of` . Objects have keys you iterate over — `for...in` . Everything else works like PHP.

Try it yourself

Given `const scores = { alice: 85, bob: 72, charlie: 91 }` , log each name and their grade letter (A for 90+, B for 80-89, C for 70-79, F below).

Episode 5 — Creating Functions

TL;DR

Functions are **first-class values** in JS — you can assign them, pass them, return them. Three ways to create them: declarations (hoisted), expressions (not hoisted), and arrow functions (compact, no own `this`). IIFEs wrap code to protect it from leaking globals.

PHP → JS differences

- **Functions are values** — PHP's callable concept exists but is clunkier
- **Hoisting is a JS-specific behaviour** — function declarations move to the top of their scope automatically
- **Arrow functions** — no direct PHP equivalent; PHP's `fn()` is similar but limited

Concepts, explained

Function declaration (hoisted)

```
// This works — declarations are hoisted
sayHi();

function sayHi() {
  console.log('hi');
```

```
}
```

Function expression (NOT hoisted)

```
// This throws – expressions aren't hoisted
// sayBye(); ← ReferenceError

const sayBye = function () {
  console.log('bye');
};

sayBye(); // works here, after assignment
```

Arrow functions

```
// Single expression, implicit return
const multiply = (a, b) => a * b;

// Block body, explicit return
const divide = (a, b) => {
  if (b === 0) throw new Error('Cannot divide by zero');
  return a / b;
};

// Single parameter – parens optional
const square = x => x * x;

// No parameters – parens required
const now = () => Date.now();
```

Every function returns *something*

If you don't `return` explicitly, JS returns `undefined`.

```
function noReturn() {
  const x = 1;
}

console.log(noReturn()); // undefined
```

Anonymous functions

Functions without a name — commonly used as callbacks.

```
setTimeout(function () {
  console.log('delayed');
}, 1000);

// Arrow version
setTimeout(() => console.log('delayed'), 1000);
```

IIFE — Immediately Invoked Function Expression

Wrap code to protect it from leaking into global scope. The surrounding parens are **required** — without them, JS tries to parse `function` as a declaration and crashes.

```
(function () {
  const secret = 'hidden';
  console.log(secret);
})();

// Arrow version
(() => {
  const secret = 'hidden';
  console.log(secret);
})();
```

Gotchas & warnings

- **Function expressions vs declarations.** Calling a `const fn = function(){}` before assignment throws. Calling a `function fn(){}` before declaration works (hoisting).
- **Forgetting the IIFE parens.** `function(){}()` is a syntax error. Must be `(function(){}())` — the wrapper turns it into an expression.
- **Arrow functions have no own `this`** — covered in Ep 13. This will bite you in event handlers.

Mental model

Functions are just values that happen to be callable. You can store them in variables, pass them around, and invoke them later. Declarations hoist; expressions and arrows don't. IIFEs are "do this once and don't pollute anything."

Try it yourself

Write an IIFE that defines a counter variable inside itself, and exposes a single function via a global assignment (e.g. `window.tick = ...`) that increments and logs the counter. Prove the variable is unreachable from outside.

Episode 6 — Scoping Out The Variables

TL;DR

`var` is function-scoped (PHP-like), `let` is block-scoped (modern), `const` is block-scoped and non-reassignable. All three are useful for different reasons, but `const` first, `let` if you must reassign, `var` almost never.

PHP → JS differences

- **Block scoping** (`{ }`) is a JS concept `let` / `const` introduced; PHP doesn't have it in the same way
- **Global variables are inspectable and mutable** from the browser console — a security reality, not just a bug
- **`const` on objects/arrays fixes the binding, not the contents** — the reference can't change but the insides can

Concepts, explained

Generating a random integer

```
const roll = Math.floor(Math.random() * 10) + 1; // 1 to 10
```

Browser's `prompt()`

```
const answer = prompt('Enter a number:');  
// returns a string or null if cancelled
```

Converting string to number

```
Number('42'); // 42  
Number('3.14'); // 3.14  
Number('abc'); // NaN  
Number(''); // 0 (careful!)  
  
isNaN(Number('abc')); // true
```

`var` — function-scoped

```
function demo() {  
  if (true) {  
    var x = 1;  
  }  
  console.log(x); // 1 — leaks out of the if block  
}
```

`let` — block-scoped

```
function demo() {  
  if (true) {  
    let x = 1;  
  }  
  console.log(x); // ReferenceError — x doesn't exist here  
}
```

`const` — block-scoped, can't be reassigned

```
const pi = 3.14;  
pi = 3.14159; // TypeError: Assignment to constant variable
```

But for objects and arrays, the **contents** are still mutable:

```
const user = { name: 'Ben' };  
user.name = 'Benjamin'; // ALLOWED — mutating contents  
user.age = 23; // also allowed  
// user = {}; // NOT allowed — reassigning the binding
```

```
const items = [1, 2, 3];  
items.push(4); // ALLOWED  
items[0] = 99; // ALLOWED  
// items = []; // NOT allowed
```

Devtools reality check

Open the browser console on any site and you can **read and modify** any global variable. Client-side

JS is never trustworthy for anything security-sensitive.

```
// If the site has:  
var apiKey = 'secret-123';  
  
// An attacker opens DevTools and runs:  
console.log(apiKey); // sees it  
apiKey = 'hijacked'; // changes it
```

This is why IIFEs matter — they hide internals.

Gotchas & warnings

- **const is not "immutable."** It prevents reassignment of the binding, not mutation of the value. `const arr = []; arr.push(1)` is valid.
- **var leaks out of blocks.** In a `for` or `if`, a `var` escapes. Use `let`.
- **Nothing in the browser is secret.** Anything your client-side JS can do, a user in DevTools can redo or change.

Mental model

Think of `let` and `const` as block-scoped (they live in their nearest `{ }`). Think of `var` as function-scoped. Default to `const`; switch to `let` only when you *need* to reassign.

Try it yourself

Write three loops — one with `var i`, one with `let i`, one with `const i`. Observe which succeed, which fail, and when. Explain `const`'s failure mode.

Episode 7 — The Basics Of JavaScript Arrays

TL;DR

Arrays are dynamic, index-based, and have convenient methods like `.push`, `.indexOf`, `.join`, and `.length`. The array literal `[1, 2, 3]` is the idiomatic way to create one. `Array.of()` and `Array.from()` handle edge cases the constructor mishandles.

PHP → JS differences

- **No associative arrays** — use objects for key/value

- `.push(x)` instead of `$arr[] = x` — a method, not a syntax
- `.length` is a property, not a function — don't call it
- Arrays grow and shrink dynamically — no fixed size

Concepts, explained

Creation

```
const a = new Array();           // []
const b = new Array(5);          // [empty × 5] – 5 empty slots!
const c = new Array(1, 2, 3);    // [1, 2, 3]
const d = [1, 2, 3];             // literal – preferred

const e = Array.of(5);           // [5] – single-element array
const f = Array.from('hello');   // ['h','e','l','l','o']
```

Note the trap: `new Array(5)` makes an array of 5 empty slots, `Array.of(5)` makes `[5]`.

Common methods

```
const nums = [1, 2, 3];

nums.push(4);           // [1, 2, 3, 4] – append
nums.push(5, 6);        // [1, 2, 3, 4, 5, 6] – multiple
nums.indexOf(3);         // 2
nums.indexOf(99);        // -1 (not found)
nums.length;            // 6 – property, not function
nums.join(', ');        // '1, 2, 3, 4, 5, 6'
```

`const` on arrays — binding is fixed, contents are not

```
const history = [];
history.push('guess 1'); // works
history.push('guess 2'); // works
history.length = 0;      // reset in place (sets length to 0, truncates)

// history = [];         // would throw – can't reassign
```

Using `.length` instead of a separate counter

```
// Verbose
let count = 0;
count++; // each time you push

// Better: let the array itself be the truth
console.log(history.length);
```

PHP analogue

```
$nums = [1, 2, 3];
$nums[] = 4;           // push
count($nums);          // length
array_search(3, $nums); // indexOf
implode(', ', $nums);  // join
```

Gotchas & warnings

- `new Array(5)` vs `Array.of(5)` . The first creates five empty slots; the second creates `[5]` .
- `.length` is a property. `arr.length()` throws — no parens.
- `length` is writable. Setting `arr.length = 0` truncates the array. Setting it higher adds empty slots.

Mental model

Arrays in JS are ordered, zero-indexed, dynamic lists. Everything you do with them is via methods (`.push` , `.indexOf` , `.join`) — there's no special array syntax like PHP's `[]` pushing.

Try it yourself

Create an empty array. Push three guesses. Join them with `" | "` . Truncate the array to empty by setting `.length = 0` . Verify with `console.log` .

Episode 8 — Intro To Objects

TL;DR

Objects are JS's equivalent of PHP associative arrays but richer — they can hold methods too. Build them with `{}` literals, access properties with `.` or `[...]` , and know that methods are just properties that happen to reference functions.

PHP → JS differences

- `{}` is the idiomatic way to build key/value data — not arrays
- Methods are just function-valued properties — no separate concept
- `this` refers to the object — same idea as PHP
- Two access syntaxes — dot and bracket

Concepts, explained

Creation — literal vs constructor

```
const a = new Object(); // rare
const b = {};           // preferred
```

Adding properties — dynamic by default

```
const person = {};
person.firstName = 'Benjamin';
person['lastName'] = 'Shaw';
```

Property access — dot vs bracket

```
person.firstName;           // 'Benjamin'
person['firstName'];        // same

const key = 'firstName';
person[key];                // 'Benjamin' — must use brackets for variable keys
// person.key              // WRONG — looks for a literal "key" property
```

Initializing with an object literal

```
const person = {
  firstName: 'Benjamin',
  lastName: 'Shaw',
  age: 23,
};
```

Methods as function-valued properties

```
const person = {
  firstName: 'Benjamin',
  lastName: 'Shaw',
  getFullName: function () {
    return this.firstName + ' ' + this.lastName;
  },
};

person.getFullName(); // 'Benjamin Shaw'
```

Shorthand method syntax

```
const person = {
  firstName: 'Benjamin',
  lastName: 'Shaw',
  getFullName() {
    return `${this.firstName} ${this.lastName}`;
  },
};
```

this refers to the object

```
const game = {
  score: 0,
  increment() {
    this.score++;
  },
};

game.increment();
game.increment();
console.log(game.score); // 2
```

Factory via IIFE

```
const game = (function () {
  let score = 0;

  return {
    play() {
      score++;
      console.log(`Score: ${score}`);
    }
  };
})();
```

```

    },
    reset() {
      score = 0;
    },
  });
})();

game.play(); // Score: 1
game.play(); // Score: 2
game.reset();

```

JSON vs object literal — subtle difference

```

// Valid object literal — bare identifier keys
const obj = { name: 'Ben', age: 23 };

// JSON — requires quoted string keys
const json = '{"name":"Ben","age":23}';

```

Resetting an array in place

```

const history = [1, 2, 3];
history.length = 0;
console.log(history); // []

```

Gotchas & warnings

- **Dot notation won't accept a variable.** `obj.key` vs `obj[key]` are different things.
- **this inside an arrow function is different.** (Covered in Ep 13.) Use traditional `function` or shorthand methods on objects.
- **JSON ≠ JS object literal.** JSON requires double-quoted keys; object literals don't. They look similar but aren't interchangeable.

Mental model

An object is a bag of key/value pairs where values can be anything — including other functions. Methods are just properties that happen to point to a function. `this` is "the object I was called on."

Try it yourself

Build an object `account` with `balance` and two methods `deposit(amount)` and `withdraw(amount)`. Guard against negative amounts. Test it.

Episode 9 — Using Objects As Parameters

TL;DR

Instead of passing five positional args to a function (and forcing callers to remember their order), pass a single options object. This is the dominant pattern in modern JS frameworks — React, Vue, Svelte, Alpine all use it.

PHP → JS differences

- **Options-object pattern is the norm in JS** — PHP uses it less, tends to rely on positional args or named arguments (PHP 8+)
- **Default parameter values** have existed in both for years, same idea

Concepts, explained

The problem — positional args

```
// Caller has to remember the order
function play(minRange, maxRange, maxAttempts) {
  // ...
}

play(1, 10, 3);
play(5, 50, 10);
```

If you want to skip `maxRange` and just set `maxAttempts`, you can't.

Options object pattern

```
function play(options) {
  const minRange = typeof options.minRange === 'undefined' ? 1 : options.minRange;
  const maxRange = typeof options.maxRange === 'undefined' ? 10 : options.maxRange;
  const maxAttempts = typeof options.maxAttempts === 'undefined' ? 3 : options.maxAttempts;

  console.log(minRange, maxRange, maxAttempts);
}

play({ maxAttempts: 5 }); // 1, 10, 5
play({ minRange: 1, maxRange: 100 }); // 1, 100, 3
```

Callers pick and choose which properties to set.

Default parameter values (simpler cases)

```
function play(minRange = 1, maxRange = 10, maxAttempts = 3) {
  // ...
}

play(); // 1, 10, 3
play(5); // 5, 10, 3
play(5, 50); // 5, 50, 3
```

But this brings back the positional problem for complex cases.

Refactor — accept `options = {}`

```
function play(options = {}) {
  const minRange = typeof options.minRange === 'undefined' ? 1 : options.minRange;
  const maxRange = typeof options.maxRange === 'undefined' ? 10 : options.maxRange;
  const maxAttempts = typeof options.maxAttempts === 'undefined' ? 3 : options.maxAttempts;
  console.log({ minRange, maxRange, maxAttempts });
}
```

```
play(); // works – options defaults to {}
play({ maxAttempts: 5 }); // flexible
```

Dynamic options

```
const settings = { minRange: 1 };
if (difficulty === 'hard') settings.maxAttempts = 5;
if (largeRange) settings.maxRange = 100;

play(settings);
```

This is why the pattern wins — you can **build up** the options object conditionally before calling.

Gotchas & warnings

- **Forgetting `= {}` on the parameter.** If you call `play()` without an arg and the param is `options`, then `options.minRange` throws "Cannot read property ... of undefined". Always `options = {}`.
- **Default value assignment runs per call.** Don't default to a mutable shared object unless you mean it.

Mental model

When a function grows past 2-3 params, stop passing positional args. Pass one object. Callers get named, skippable, dynamically-built configuration.

Try it yourself

Refactor `createUser(firstName, lastName, email, age, role)` to accept a single options object with sensible defaults.

Episode 10 — Destructuring Objects

TL;DR

Destructuring is syntactic sugar for extracting properties into variables. Combined with defaults and the nullish coalescing operator (`??`), it collapses the verbose options-object boilerplate from Ep 9 into one clean line.

PHP → JS differences

- **Destructuring** is a built-in JS syntax; PHP has it via `[...] = $array`, but JS is more expressive with objects
- **?? nullish coalescing** — same operator, same meaning (PHP 7+ and modern JS)

Concepts, explained

Nullish coalescing ??

Returns the right side only if the left side is `null` or `undefined`. Does NOT trigger on `0`, `''`, or `false`.

```
const a = null ?? 'default'; // 'default'
const b = undefined ?? 'default'; // 'default'
const c = 0 ?? 'default'; // 0 ← keeps 0!
const d = '' ?? 'default'; // '' ← keeps empty string
```

Compare to `||` which triggers on any falsy:

```
const bad = 0 || 'default'; // 'default' — PROBABLY WRONG
const good = 0 ?? 'default'; // 0 — usually what you want
```

Object destructuring — extract into variables

```
const options = { minRange: 1, maxRange: 10, maxAttempts: 3 };
const { minRange, maxRange, maxAttempts } = options;
console.log(minRange, maxRange, maxAttempts); // 1 10 3
```

With defaults during destructuring

```
const options = { minRange: 5 };
const { minRange = 1, maxRange = 10, maxAttempts = 3 } = options;
console.log(minRange, maxRange, maxAttempts); // 5 10 3
```

Destructuring in function signatures

```
function play({ minRange = 1, maxRange = 10, maxAttempts = 3 } = {}) {
  console.log({ minRange, maxRange, maxAttempts });
}

play(); // 1 10 3 (defaults)
play({ maxAttempts: 5 }); // 1 10 5
play({ minRange: 1, maxRange: 100 }); // 1 100 3
```

The `= {}` at the end is **critical** — without it, calling `play()` with no args throws because you can't destructure `undefined`.

Compare — before and after

Before (Ep 9):

```
function play(options = {}) {
  const minRange = typeof options.minRange === 'undefined' ? 1 : options.minRange;
```

```
const maxRange = typeof options.maxRange === 'undefined' ? 10 : options.maxRange;
const maxAttempts = typeof options.maxAttempts === 'undefined' ? 3 : options.maxAttempts;
// ...
}
```

After (Ep 10):

```
function play({ minRange = 1, maxRange = 10, maxAttempts = 3 } = {}) {
  // ...
}
```

Same behaviour, a fraction of the code.

Renaming during destructure

```
const user = { first: 'Ben', last: 'Shaw' };
const { first: firstName, last: lastName } = user;
console.log(firstName, lastName); // Ben Shaw
```

Gotchas & warnings

- **Missing `= {}` at the end of a destructured param.** Calling with no arg throws "Cannot destructure property ... of undefined." Always default to `{}`.
- **`??` vs `||`.** Use `??` when `0`, `''`, or `false` are legitimate values. Use `||` only when any falsy should trigger the fallback.
- **Destructuring doesn't copy.** For nested objects, you're still referencing the same underlying object.

Mental model

Destructuring is the "unpack" operator for objects. Combined with defaults and `??`, it turns options-bag boilerplate into a single readable line.

Try it yourself

Rewrite `function createUser(options) { ... }` using destructuring with defaults. Handle the `options === undefined` case. Test with no args, partial args, full args.

Episode 11 — Factories And Classes

TL;DR

Two ways to build repeatable objects with shared behaviour: **factory functions** (return a fresh object each call, use closures for private state) and **classes** (share methods across instances, use `#` for truly private fields). Factories are lighter but duplicate methods per instance; classes are memory-efficient for many instances.

PHP → JS differences

- **Factory functions** aren't a formal PHP pattern but you can build them with a function returning `new stdClass()` + callables
- **Classes are public by default** — PHP defaults to public too, but explicit `private` / `protected` is more idiomatic there
- **Private fields use `#`** (not `private` keyword) — syntactic parallel to PHP but visually different
- **No class constants** like PHP's `const MAX = 5;`

Concepts, explained

Factory function

```
const createGame = function (options = {}) {
  const { minRange = 1, maxRange = 10, maxAttempts = 3 } = options;
  let attempts = 0;

  return {
    play() {
      attempts++;
      console.log(`Attempt ${attempts}, range ${minRange}-${maxRange}`);
    },
    reset() {
      attempts = 0;
    },
  };
};

const easy = createGame({ maxRange: 10 });
const hard = createGame({ maxRange: 100 });

easy.play(); // Attempt 1, range 1-10
hard.play(); // Attempt 1, range 1-100
```

Each call returns a **new** object with its own closed-over `minRange`, `maxRange`, `attempts`.

Property shorthand

When the variable name matches the property name:

```
const minRange = 1, maxRange = 10;

// Long form
const opts1 = { minRange: minRange, maxRange: maxRange };

// Shorthand — same thing
const opts2 = { minRange, maxRange };
```

Factory drawback — methods duplicated per instance

Every `createGame()` call creates fresh copies of `play` and `reset`. If you make 10,000 games, you have 20,000 function objects in memory.

Class syntax

```
class Game {
  constructor(options = {}) {
    const { minRange = 1, maxRange = 10, maxAttempts = 3 } = options;
    this.minRange = minRange;
    this.maxRange = maxRange;
    this.maxAttempts = maxAttempts;
    this.attempts = 0;
  }

  play() {
    this.attempts++;
    console.log(`Attempt ${this.attempts}, range ${this.minRange}-${this.maxRange}`);
  }

  reset() {
    this.attempts = 0;
  }
}

const easy = new Game({ maxRange: 10 });
easy.play();
```

Methods live on the class's prototype — **shared across all instances**. 10,000 games share one `play` function.

Private fields with `#`

```
class Game {
  #minRange;
  #maxRange;
  #attempts = 0;

  constructor({ minRange = 1, maxRange = 10 } = {}) {
    this.#minRange = minRange;
    this.#maxRange = maxRange;
  }

  play() {
    this.#attempts++;
    return `Attempt ${this.#attempts} in range ${this.#minRange}-${this.#maxRange}`;
  }
}

const game = new Game();
game.play();
// game.#attempts      ← SyntaxError — private, not accessible from outside
```

PHP analogue

```
class Game {
  private int $attempts = 0;

  public function __construct(
    private int $minRange = 1,
    private int $maxRange = 10,
  ) {}

  public function play(): string {
    $this->attempts++;
    return "Attempt {$this->attempts}";
  }
}
```

Gotchas & warnings

- **Factory functions use more memory.** Each instance duplicates every method. Fine for 10 instances, bad for 10,000.
- `#private` syntax is not `private` keyword. Easy to type the wrong thing.

- **No class constants.** Work around with `static` members (Ep 12) or module-level `const`.

Mental model

Factories return a new object each call with closures protecting private state. Classes define a blueprint where methods are shared across all instances. Use factories for data-shaped things you build a few of; use classes for behaviour-heavy things you'll build many of.

Try it yourself

Write both versions of a `Counter` — once as a factory returning `{ increment, reset, get value() { ... } }`, once as a class with `#count` and `increment()`. Compare the code.

Episode 12 — Static Members And Property Accessors

TL;DR

`static` methods live on the class itself, not on instances — you call them as `ClassName.method()`. Property accessors (`get` / `set`) let you treat a method-backed value like a plain property, wrapped with validation.

PHP → JS differences

- **Static access syntax** is `ClassName.method()`, not `self::method()`
- **Property accessors** exist natively in JS (class syntax); PHP 8.4 added property hooks which are analogous
- **throw** can throw any value, not just `Throwable` instances

Concepts, explained

Static methods

```
class Game {
  static initRangeValues({ value, lowerBounds, upperBounds = 0 } = {}) {
    if (isNaN(value)) throw new Error('value must be numeric');
    if (value < lowerBounds) return lowerBounds;
    if (upperBounds && value > upperBounds) return upperBounds;
    return value;
  }
}

Game.initRangeValues({ value: 5, lowerBounds: 1, upperBounds: 10 }); // 5
```

```
Game.initRangeValues({ value: 99, lowerBounds: 1, upperBounds: 10 }); // 10 (clamped)
```

Calling static from inside the class — use the class name

```
class Game {
  static initRangeValues(opts) { /* ... */ }

  constructor({ minRange }) {
    // Not `self` — use the class name directly
    this.minRange = Game.initRangeValues({ value: minRange, lowerBounds: 0 });
  }
}
```

Throwing anything

```
// Plain object
throw { message: 'value must be numeric' };

// More semantic — an Error instance has .stack, .name, etc.
throw new Error('value must be numeric');
```

Prefer `new Error(...)` unless you have a reason not to.

Property accessors (`get` / `set`)

```
class Game {
  #minRange;
  #maxRange = 10;

  constructor({ minRange = 1 } = {}) {
    this.minRange = minRange; // triggers the setter
  }

  get minRange() {
    return this.#minRange;
  }

  set minRange(value) {
    this.#minRange = Game.initRangeValues({
      value,
      lowerBounds: 0,
      upperBounds: this.#maxRange,
    });
  }

  static initRangeValues({ value, lowerBounds, upperBounds = 0 } = {}) {
    if (isNaN(value)) throw new Error('value must be numeric');
    if (value < lowerBounds) return lowerBounds;
    if (upperBounds && value > upperBounds) return upperBounds;
    return value;
  }
}

const game = new Game({ minRange: 5 });
console.log(game.minRange); // 5 — triggers getter
game.minRange = 999; // triggers setter — clamps to maxRange (10)
console.log(game.minRange); // 10
```

From the outside, `game.minRange` looks like a plain property, but it's running validation under the hood.

Accessors on object literals too

```
const person = {
  _firstName: 'Ben',
  get firstName() {
    return this._firstName;
  },
  set firstName(value) {
    if (typeof value !== 'string') throw new Error('must be a string');
    this._firstName = value;
  },
};
```

```
person.firstName = 'Benjamin'; // triggers setter
console.log(person.firstName); // triggers getter
```

Gotchas & warnings

- `ClassName.staticMethod()` **inside the class feels weird** coming from PHP's `self::`. Just know that JS does it that way.
- **Throwing a plain object vs an Error.** Plain objects don't get a stack trace. Use `new Error(...)`.
- **Accessors can loop forever** if you accidentally use the public property name inside the getter:
`get minRange() { return this.minRange; }` — infinite recursion. Use a private backing field (`#minRange`).

Mental model

`static` = "this belongs to the class, not to any instance." `get` / `set` = "this looks like a property but runs code on read/write." Together they give you validated, ergonomic APIs.

Try it yourself

Add a `get difficulty()` accessor to a `Game` class that returns `'easy'` if `maxRange - minRange < 20`, `'medium'` up to 50, else `'hard'`.

Episode 13 — Context

TL;DR

`this` in JS isn't fixed — it depends on **how** the function is called, not where it was defined. Detaching a method from its object loses `this`. Use `.bind()` to permanently glue a function to a context.

PHP → JS differences

- PHP's `$this` **is stable** — it's always the current object inside a method
- JS's `this` **is dynamic** — the same method can have different `this` depending on how it's invoked
- JS has `bind()`, `call()`, `apply()` for explicit context — no PHP equivalent

Concepts, explained

`this` is set by the caller, not the definition

```
const game = {
  name: 'Guessing Game',
  describe() {
    return `I am ${this.name}`;
  },
};

game.describe(); // 'I am Guessing Game' — `this` is `game`
```

Detaching a method breaks `this`

```
const describe = game.describe;
describe(); // 'I am undefined' (or throws in strict mode)
```

Why: you copied the function object. When you call `describe()` standalone, there's no "receiver" — `this` is `undefined`.

Attaching the method to a different object

```
const other = { name: 'Other Game' };
other.describe = game.describe;
other.describe(); // 'I am Other Game' — `this` is now `other`
```

Explicit binding with `.bind()`

```
const boundDescribe = game.describe.bind(game);
boundDescribe(); // 'I am Guessing Game' — permanently bound to `game`
```

`.bind()` returns a **new function** that has `this` permanently wired to whatever you passed.

When this matters — event handlers

```
class Game {
  constructor() {
    this.score = 0;
  }
  handleClick() {
    this.score++;
    console.log(this.score);
  }
}

const game = new Game();

// WRONG — when the event fires, `this` will be the DOM element, not the game
button.addEventListener('click', game.handleClick);

// RIGHT — bind it to the game
button.addEventListener('click', game.handleClick.bind(game));

// ALSO RIGHT — arrow functions capture `this` lexically
button.addEventListener('click', () => game.handleClick());
```

The keyboard analogy

A method detached from its object is like a keyboard unplugged from a computer — it still types characters, but there's no machine on the other end. `.bind()` is the cable that wires it back to a specific machine.

Gotchas & warnings

- **Passing `obj.method` as a callback** almost always breaks `this`. This is the #1 React/DOM bug for JS newcomers.
- **Arrow functions have NO own `this`** — they inherit from the enclosing scope. This is usually a feature in callbacks but a bug inside object methods.
- **`.bind()` returns a new function.** It doesn't modify the original.

Mental model

`this` is a hidden argument passed based on **how the function is called**. `obj.method()` — `this` is `obj`. `obj.method()` (detached) — `this` is `undefined`. `method.bind(x)()` — `this` is `x`. Arrow functions skip this whole system and inherit from their enclosing scope.

Try it yourself

Create an object with a method that uses `this`. Detach it, observe it break. Rebind with `.bind()`. Observe it work. Then rewrite as an arrow function and predict the result.

Episode 14 — Finding Elements

TL;DR

The `document` object is your gateway to the DOM. Three main ways to find elements: `getElementById` (fastest), `querySelector` (flexible, first match), `querySelectorAll` (all matches). Put your `<script>` at the end of `<body>` or your JS will run before the DOM exists.

PHP → JS differences

- **DOM manipulation is a browser thing**, not PHP — though the idea of navigating a document tree will be familiar from XML parsing or DOM extensions
- **Null checks matter** — `getElementById` returns `null` if nothing found, not an empty object

Concepts, explained

`getElementById`

```
const title = document.getElementById('gameTitle');
title.innerHTML = 'Welcome!';
```

Returns the matching element or `null`.

querySelector — first match for any CSS selector

```
const list = document.querySelector('ul.list-disc');
const firstBtn = document.querySelector('button');
const submit = document.querySelector('form#signup button[type="submit"]');
```

querySelectorAll — all matches

```
const headings = document.querySelectorAll('h2, h3');
// Returns a NodeList – not an array, but iterable
headings.forEach(h => console.log(h.textContent));
```

Setting content

```
const el = document.getElementById('status');
el.innerHTML = '<strong>Loading...</strong>';
```

`innerHTML` parses HTML. For plain text, use `textContent` to avoid XSS risks.

The script placement gotcha

Bad:

```
<head>
  <script src="app.js"></script>
</head>
<body>
  <h1 id="title">Hello</h1>
</body>
```

`getElementById('title')` returns `null` — the script ran before `<h1>` was parsed.

Good:

```
<body>
  <h1 id="title">Hello</h1>
  <script src="app.js"></script>
</body>
```

Or use `<script defer src="...">` in the head — `defer` waits until the DOM is parsed.

Gotchas & warnings

- `getElementById` returns `null` if no match. Calling a method on `null` throws.
- `querySelectorAll` returns a **NodeList**, not an array. Some array methods work (`.forEach`), others don't (`.map` , `.filter`). Convert with `[...nodeList]` or `Array.from()` .
- **Script placement.** Top-of-head scripts run before DOM exists. End-of-body or `defer` fixes it.
- **innerHTML with user input is an XSS hole.** Use `textContent` unless you trust the source.

Mental model

The browser builds a tree of nodes from your HTML. `document` is the root. You navigate to elements by

ID, CSS selector, or by walking the tree. Elements have properties like `innerHTML` you can read and write.

Try it yourself

Create an HTML page with three `<h2>` s. In JS, select them all with `querySelectorAll` and change their text to include their index.

Episode 15 — Creating Elements And Content

TL;DR

Build DOM nodes with `document.createElement()` and `createTextNode()`, then attach with `appendChild()`. For performance, build a `DocumentFragment` in memory and append it once — the browser only redraws once. Modern frameworks handle all this for you, but knowing the raw API is foundational.

PHP → JS differences

- **DOM creation is browser-specific** — no PHP parallel
- **Appending to an element adds to the end** — doesn't replace existing children
- Use `replaceChildren()` to wipe and rewrite

Concepts, explained

Create an element and attach it

```
const li = document.createElement('li');
const text = document.createTextNode('New item');
li.appendChild(text);

const list = document.getElementById('itemList');
list.appendChild(li);
```

Document fragments — batch for performance

```
const frag = document.createDocumentFragment();
for (let i = 1; i <= 100; i++) {
  const li = document.createElement('li');
  li.textContent = `Item ${i}`;
  frag.appendChild(li);
}
```

```
// One DOM insertion instead of 100
document.getElementById('itemList').appendChild(frag);
```

The browser only redraws once instead of 100 times.

Replace vs append

```
const list = document.getElementById('itemList');
// Append – adds to existing content
list.appendChild(li);

// Replace – wipe existing children, then add
list.replaceChildren(); // empty the list
list.appendChild(newLi); // add the new one

// Or pass multiple children to replaceChildren
list.replaceChildren(newLi1, newLi2, newLi3);
```

Quick-and-dirty with `innerHTML`

```
const list = document.getElementById('itemList');
// Append
list.innerHTML += '<li>New item</li>'; // works, but parses the whole innerHTML again
// Replace
list.innerHTML = '<li>Only item</li>';
```

`innerHTML` is concise but has XSS risk if interpolating user input — sanitize or use `textContent`.

Gotchas & warnings

- **Append doesn't replace.** Calling `appendChild` over and over stacks new items on top of whatever's already there.
- `innerHTML +=` **rebuilds the entire subtree.** Fine for small updates, slow for hot paths.
- **Don't interpolate user input into `innerHTML`.** XSS risk — use `textContent` or sanitize.
- **DocumentFragment "disappears" after append** — it's designed that way; the fragment's children move to the new parent.

Mental model

DOM is a tree of nodes. You make new nodes with `createElement` / `createTextNode`, then attach them somewhere. For bulk inserts, build offstage in a `DocumentFragment` and drop it in once.

Try it yourself

Build a list of 10 `<li>` items with `createElement` and append them to an existing `<ul>`. Time the naive version (loop with `appendChild`) vs the fragment version.

Episode 16 — Event Basics

TL;DR

Use `addEventListener` over inline `onclick` attributes. Events deliver an `event` object with info like `target`, `altKey`, mouse coordinates. Be careful with `this` in callbacks — if you pass `obj.method` directly, `this` will be the element, not your object.

PHP → JS differences

- **Events are a browser/runtime concept**, no direct PHP equivalent
- **Inline `onclick` is discouraged** — modern style is `addEventListener`

Concepts, explained

DOM Level 0 — inline (avoid)

```
<button onclick="alert('hi')">Click</button>
```

Drawbacks: mixes JS into HTML, hard to maintain, race conditions if script loads late.

DOM Level 2 — `addEventListener` (preferred)

```
const btn = document.getElementById('submitBtn');
btn.addEventListener('click', function (event) {
  console.log('clicked', event);
});
```

Notice: the event name is `'click'`, **not** `'onclick'` — no `on` prefix.

Listening on `window`

```
window.addEventListener('load', () => {
  console.log('page fully loaded');
});
window.addEventListener('resize', () => {
  console.log(window.innerWidth);
});
```

The `window` is the global object in the browser — every global variable is a property of it.

The event object

```
btn.addEventListener('click', (event) => {
  console.log(event.target); // the element that was clicked
  console.log(event.altKey); // true if Alt was held
  console.log(event.ctrlKey); // Ctrl
  console.log(event.clientX); // mouse X
  console.log(event.clientY); // mouse Y
});
```

```
});
```

Context (`this`) in event handlers

Inside a handler, `this` is the element the listener was attached to.

```
btn.addEventListener('click', function () {  
  console.log(this); // the button  
});
```

So passing an object's method directly breaks `this`:

```
const game = {  
  score: 0,  
  play() {  
    this.score++; // `this` is NOT `game` when called as a handler!  
  },  
};  
  
// WRONG  
btn.addEventListener('click', game.play);  
  
// RIGHT — bind  
btn.addEventListener('click', game.play.bind(game));  
  
// ALSO RIGHT — arrow wrapper  
btn.addEventListener('click', () => game.play());
```

Gotchas & warnings

- **The event name has no `on` prefix.** `addEventListener('click', ...)`, not `'onclick'`.
- **`this` inside a regular-function handler is the element**, not whatever object you "meant." Bind or wrap in an arrow.
- **Removing listeners requires the same function reference.** You can't `removeEventListener('click', () => {...})` — the arrow function is a new reference. Store the handler in a variable if you need to remove it later.

Mental model

Events are messages the browser fires at elements. You attach listeners to receive them. Every listener gets an `event` object with the details, and `this` (in a regular function) is the element that caught the event.

Try it yourself

Add a click listener to a button that alerts the user which modifier keys (Alt, Ctrl, Shift) were held when they clicked.

Episode 17 — Form Basics

TL;DR

Form controls work whether or not they're in a `<form>`, but semantic HTML prefers one. Use `event.preventDefault()` to stop default form submission. The form-level `submit` event combined with `event.submitter` lets you handle any button's click from a single listener.

PHP → JS differences

- **Forms submit to a URL by default** — in a JS-first app, you usually `preventDefault` and handle submission via JS

Concepts, explained

Reading and writing input values

```
const input = document.getElementById('titleInput');

console.log(input.value); // read
input.value = 'New title'; // write
input.value = ''; // clear
```

Required-field check — empty string is falsy

```
const title = input.value.trim();
if (!title) {
  alert('Title is required');
  return;
}
```

Preventing form submission

```
const form = document.getElementById('myForm');

form.addEventListener('submit', (event) => {
  event.preventDefault(); // stops the browser from navigating / POSTing
  // ... handle in JS
});
```

Identifying which button submitted the form

When multiple buttons exist (`Save` , `Save and Continue` , `Delete`), use `event.submitter` :

```
form.addEventListener('submit', (event) => {
  event.preventDefault();

  const action = event.submitter.name || event.submitter.id;

  if (action === 'saveBtn') {
    // ...
  } else if (action === 'deleteBtn') {
    // ...
  }
});
```

Much cleaner than attaching click listeners to every button.

Why `submit` beats per-button `click`

```
// Verbose – one listener per button
saveBtn.addEventListener('click', handleSave);
deleteBtn.addEventListener('click', handleDelete);
cancelBtn.addEventListener('click', handleCancel);

// Better – one listener on the form, branch inside
form.addEventListener('submit', (event) => {
  event.preventDefault();
  switch (event.submitter.id) {
    case 'saveBtn': handleSave(); break;
    case 'deleteBtn': handleDelete(); break;
    case 'cancelBtn': handleCancel(); break;
  }
});
```

Gotchas & warnings

- **Forgetting `preventDefault`**. The page reloads / navigates, your JS state dies.
- **Buttons inside forms submit by default.** Set `type="button"` on non-submit buttons, or always `preventDefault` and branch on `event.submitter`.
- **`input.value` is always a string.** Even `type="number"` gives you a string.

Mental model

Think of the `submit` event as the single entry point. Every button press that would submit the form flows through there. `event.submitter` tells you which button triggered it. This beats scattering click handlers across buttons.

Try it yourself

Build a form with three buttons (Save, Reset, Delete). Handle all three via a single `submit` listener using `event.submitter`.

Episode 18 — Working With "Select" Elements

TL;DR

`select.value` gives you the selected option's value. For more detail use `selectedIndex`, `options`, or `selectedOptions`. Use `data-*` attributes to attach custom data to elements — access with `element.dataset.someKey` (camelCase).

PHP → JS differences

- **Data attributes are an HTML5 feature**, available from any JS — no PHP analog
- **Writing a helper like `getBy(sel)`** is common in JS to avoid repetitive `document.querySelector` calls

Concepts, explained

Reading a select's value

```
const select = document.getElementById('gameType');
console.log(select.value); // value of selected option
```

More detail — `selectedIndex`, `options`, `selectedOptions`

```
console.log(select.selectedIndex); // 0-based index of selection
console.log(select.options);      // HTMLCollection of all <option>s
console.log(select.options[select.selectedIndex].text); // visible text
console.log(select.selectedOptions[0]); // first selected (works for multi-select too)
```

Helper for repeated lookups

```
const getBy = (selector) => document.querySelector(selector);
const title = getBy('#gameTitle');
const list = getBy('ul.guesses');
```

Data attributes

HTML:

```
<option value="easy" data-min-range="1" data-max-range="10">Easy</option>
```

JS access — two ways:

```
// Raw attribute
const min = option.getAttribute('data-min-range'); // always a string

// dataset — camelCase, dashes dropped
const min2 = option.dataset.minRange; // "1" (still a string)
```

Mapping rule: `data-min-range` → `dataset.minRange`.

Interpolation in prompts

```
const min = select.dataset.minRange;
const max = select.dataset.maxRange;
const guess = prompt(`Guess a number between ${min} and ${max}:`);
```

Gotchas & warnings

- **dataset values are always strings.** If you stored `data-count="5"`, you get `"5"` back. Convert with `Number()` / `parseInt()` if needed.
- **selectedOptions vs options.** `options` is everything; `selectedOptions` is just what the user chose (matters for multi-selects).
- **Changing `.value` to something not in the options** silently fails — `value` resets to empty.

Mental model

Select elements are dropdowns of options. The one the user picked is accessed via `value` (fast), or via `selectedOptions` / `selectedIndex` for more detail. Data attributes are HTML's built-in way of parking custom data on elements.

Try it yourself

Build a `<select>` with three `<option>`s, each with a `data-label` attribute. On change, log the selected option's `data-label` to the console.

Episode 19 — Turn The Radio On (Or Off)

TL;DR

Radio buttons sharing a `name` form a group. Use event delegation — attach one `input` listener to the document and filter by `event.target.name` / `id`. For the currently selected radio, `form.elements.namedItem('groupName').value` is the cleanest read.

PHP → JS differences

- **Event delegation** (one listener at `document`, filter inside) is a classic DOM pattern — no PHP parallel
- **Radio semantics:** grouping by shared `name` is an HTML feature, not JS

Concepts, explained

Radios group by `name`

```
<label><input type="radio" name="gameType" value="easy"> Easy</label>
<label><input type="radio" name="gameType" value="hard"> Hard</label>
```

Event delegation — one listener, filter inside

```
document.addEventListener('input', (event) => {
  if (event.target.name === 'gameType') {
    console.log('Game type changed to:', event.target.value);
  }
});
```

The `input` event fires only when the value actually changes — clicking an already-selected radio won't spam.

Toggling UI based on selection

```
const ui = (function () {
  const easyArea = document.getElementById('easyArea');
  const hardArea = document.getElementById('hardArea');

  return {
    changeGameType(type) {
      easyArea.classList.toggle('hidden', type !== 'easy');
      hardArea.classList.toggle('hidden', type !== 'hard');
    },
  };
})();

document.addEventListener('input', (event) => {
  if (event.target.name === 'gameType') {
    ui.changeGameType(event.target.value);
  }
});
```

Reading the selected radio programmatically

Without iterating over each radio:

```
const form = document.getElementById('myForm');
const selected = form.elements.namedItem('gameType').value;
console.log(selected); // 'easy' or 'hard'
```

`form.elements.namedItem('gameType')` returns a `RadioNodeList` whose `.value` is the checked radio's value.

Gotchas & warnings

- **Forgetting to group with `name`**. Without a shared `name`, radios don't de-select each other.
- **Using `change` vs `input` events**. On radios they behave similarly; `input` is generally safer for catching every value change.
- **Caching selectors inside an IIFE** is a performance win — you look up elements once, not on every event.

Mental model

Radio buttons are a group that exposes a single value via `form.elements.namedItem(name).value`. Event delegation lets one listener handle the whole form; inside, filter by `event.target.name` or `id` to branch.

Try it yourself

Build a form with three radios (small, medium, large). Delegate input events at the document level; log the newly-selected value each time the user changes selection.

Episode 20 — Check This

TL;DR

Checkboxes expose `.checked` (boolean) — NOT `.value`. The same applies to radios. Two patterns: listen to change events and react immediately, or read `.checked` at submit time. Wrapping a checkbox read in a UI object's getter keeps intent clear.

PHP → JS differences

- `.checked` vs `.value` — a `value` on a checkbox is just the static attribute. What the user *did* is in `.checked`.

Concepts, explained

Reading checkbox state

```
const chk = document.getElementById('allowDuplicatesCheckbox');
console.log(chk.checked); // true or false
console.log(chk.value); // the static attribute — usually 'on' by default, not useful
```

Reactive pattern — listen for change

```
chk.addEventListener('change', (event) => {
  console.log('Checked?', event.target.checked);
});
```

Lazy pattern — read at submit

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const allowDups = document.getElementById('allowDuplicatesCheckbox').checked;
  startGame({ allowDuplicateGuesses: allowDups });
});
```

Getter pattern — expose checkbox state via a UI object

```
const ui = {
  get allowDuplicateGuesses() {
    return document.getElementById('allowDuplicatesCheckbox').checked;
  },
};
```



```
if (ui.allowDuplicateGuesses) {  
  // ...  
}
```

Callers don't need to know it's a DOM lookup — they just read `ui.allowDuplicateGuesses`.

PHP analogue (form handling)

```
$allowDups = isset($_POST['allowDuplicates']);
```

JS equivalent is the checkbox's `.checked`.

Gotchas & warnings

- **`.value` on a checkbox is misleading.** It's the `value` attribute, not the state. Use `.checked`.
- **Same trap applies to radios.** `radio.value` is the value attribute; `radio.checked` is whether it's selected.
- **Forgetting that `.checked` is a boolean.** Don't compare to `"true"` or `1` — use it directly in conditionals.

Mental model

For checkboxes and radios: `.value` is what the HTML says; `.checked` is what the user did. Always use `.checked`.

Try it yourself

Build a form with three checkboxes (subscribe-to-news, subscribe-to-products, subscribe-to-events). On submit, build an array of which subscriptions are active by reading each `.checked`.

Episode 21 — Filtering Input

TL;DR

Keyboard events — `keydown` fires on press (and repeats), `keyup` fires on release, `keypress` is deprecated. Use `event.key` to read the pressed key, and `event.preventDefault()` to block unwanted input. Caveat: `<input type="number">` does numeric filtering natively — only hand-roll filters when you need custom rules.

PHP → JS differences

- **Keyboard event handling is browser-only** — no PHP equivalent

Concepts, explained

Three keyboard events

```
input.addEventListener('keydown', handler); // on press, repeats while held
input.addEventListener('keyup', handler);   // on release
// input.addEventListener('keypress', handler); // deprecated — don't use
```

Use `keydown` for input filtering.

Reading the pressed key — `event.key`

```
input.addEventListener('keydown', (event) => {
  console.log(event.key);
  // 'a', 'A', '5', ' ', 'Enter', 'Backspace', 'ArrowLeft', 'Tab', ...
});
```

Numbers-only filter

```
const allowed = ['ArrowLeft', 'ArrowRight', 'Backspace', 'Delete', 'Tab'];
document.addEventListener('keydown', (event) => {
  if (event.target.id !== 'numberInput') return;

  // Allow digits
  if (event.key >= '0' && event.key <= '9') return;

  // Allow whitelisted control keys
  if (allowed.includes(event.key)) return;

  // Block everything else
  event.preventDefault();
});
```

Delegated filtering — narrow by target

```
document.addEventListener('keydown', (event) => {
  const id = event.target.id;

  if (id && id.indexOf('title') === 0) {
    // logic for elements whose id starts with "title"
  }

  if (event.target.parentNode.id === 'titleWrapper') {
    // ...
  }
});
```

Native alternative — `<input type="number">`

```
<input type="number" min="1" max="100">
```

The browser handles filtering for you. Only write manual filters when you need custom rules (e.g., allow letters but not digits).

Gotchas & warnings

- `keypress` is deprecated. Use `keydown`.
- **Pasting text bypasses `keydown`**. If a user right-click-pastes, you never see the key events — listen to `input` or `paste` events for true coverage.
- `event.key` returns strings like `'Enter'` / `'Backspace'` / `' '` — not key codes. Comparing to a single char `'a'` works directly.

Mental model

On `keydown`, check what key was pressed. If it's allowed, do nothing — the browser will type it. If not, call `preventDefault()` to block it. Only write custom filters for things HTML doesn't already do.

Try it yourself

Build an input that only accepts uppercase letters A-Z, plus Backspace / Delete / Arrow keys / Tab. Test with lowercase letters (they should be blocked) and with copy-paste.

Episode 22 — Manipulating Style

TL;DR

Three ways to change styles: `className` (destructive, replaces all classes), `style.*` (inline styles, one property at a time), and `classList` (add/remove/toggle classes — best). Prefer class-based styling — swap one class and let CSS do the work.

PHP → JS differences

- **DOM style manipulation** is browser-specific — no PHP equivalent

Concepts, explained

1. `className` — replaces the entire class attribute

```
element.className = 'new-class';  
// Destructive — drops every other class this element had
```

Avoid unless you really want to reset.

2. `style.propertyName` — inline styles

```
element.style.color = 'red';
```

```

element.style.backgroundColor = '#eee'; // camelCase – not background-color
element.style.marginTop = '10px';

// Remove
element.style.color = '';

```

CSS properties with hyphens become camelCase in JS: `background-color` → `backgroundColor`, `font-size` → `fontSize`.

3. `classList` — the sweet spot

```

element.classList.add('active');
element.classList.remove('inactive');
element.classList.toggle('hidden'); // add if missing, remove if present

element.classList.add('a', 'b', 'c'); // add multiple
element.classList.remove('a', 'b'); // remove multiple
// element.classList.toggle('a', 'b'); // NOT allowed – toggle takes only one

element.classList.contains('active'); // boolean check

```

Inline style overrides class styles

Because inline styles have higher CSS specificity.

```

<style>
  .red { color: red; }
</style>

<p class="red" style="color: blue;">I am blue, not red</p>

```

Minimize DOM writes

Each style change can trigger a browser redraw. Prefer one class swap over five individual `.style.X = ...` calls.

```

// Bad – 3 redraws
el.style.color = 'red';
el.style.backgroundColor = 'yellow';
el.style.border = '2px solid black';

// Good – 1 redraw
el.classList.add('alert');
/* in CSS:
.alert {
  color: red;
  background-color: yellow;
  border: 2px solid black;
}
*/

```

Gotchas & warnings

- `className` **replaces everything**. You'll lose Tailwind utilities, state classes, everything. Use `classList` instead.
- **CSS camelCase conversion**. `background-color` → `backgroundColor`, `z-index` → `zIndex`, `font-weight` → `fontWeight`.
- **Inline styles are hard to override**. If you're setting them via JS, remember that CSS rules won't beat them without `!important`.

Mental model

CSS classes define visual states; JS's job is to switch between them. Use `classList.toggle` / `add` / `remove` almost always. Reach for `element.style` only for truly dynamic values (like a mouse-following position).

Try it yourself

Build a button that toggles a `dark-mode` class on the `<body>`. Define CSS that changes `background` and `color` under that class. Verify one click flips everything.

Episode 23 — Event Driven Development

TL;DR

Refactor a procedural game loop (`while` + `prompt()`) into event-driven code — UI collects input, Game processes, event handler wires them. This is loose MVC. Use `parseInt(x, 10)` to convert strings to integers safely.

PHP → JS differences

- **MVC in JS-heavy apps** happens client-side — PHP usually handles it server-side
- `parseInt(x, 10)` is safer than `Number(x)` for parsing user input with possible trailing text

Concepts, explained

`parseInt` vs `Number`

```
Number('10');      // 10
Number('10abc');    // NaN - strict
parseInt('10', 10); // 10
parseInt('10abc', 10); // 10 - lenient, parses leading digits
parseInt('abc', 10); // NaN

parseFloat('3.14'); // 3.14
parseFloat('3.14m'); // 3.14 - lenient
```

Always pass the radix `10` to `parseInt` — older browsers did octal (base 8) if the string started with `0`.

Before — procedural

```
function play() {
  let attempts = 0;
  while (attempts < maxAttempts) {
    const guess = prompt('Guess a number:');
```

```

    attempts++;
    if (parseInt(guess, 10) === secret) {
      alert('You won!');
      return;
    }
  }
  alert('You lost!');
}

```

After — event-driven

```

// Game model (no UI concerns)
class Game {
  #secret;
  #history = [];
  #maxAttempts;
  #allowDuplicates;

  constructor({ maxAttempts = 5, allowDuplicates = false } = {}) {
    this.#maxAttempts = maxAttempts;
    this.#allowDuplicates = allowDuplicates;
    this.#secret = Math.floor(Math.random() * 10) + 1;
  }

  checkGuess(guess) {
    const n = parseInt(guess, 10);
    if (isNaN(n)) return { ok: false, error: 'Not a number' };
    if (!this.#allowDuplicates && this.#history.includes(n)) {
      return { ok: false, error: 'Already guessed' };
    }
    this.#history.push(n);

    if (n === this.#secret) return { ok: true, won: true };
    if (this.#history.length >= this.#maxAttempts) return { ok: true, won: false, done: true };
    return { ok: true, won: false };
  }

  get history() { return [...this.#history]; }
}

// UI object (no game logic)
const ui = {
  getGuess() {
    return document.getElementById('guessInput').value;
  },
  showFeedback(msg) {
    document.getElementById('feedback').textContent = msg;
  },
};

// Controller (wires UI and Game)
const game = new Game({ maxAttempts: 5 });
document.getElementById('guessForm').addEventListener('submit', (event) => {
  event.preventDefault();
  const result = game.checkGuess(ui.getGuess());

  if (!result.ok) { ui.showFeedback(result.error); return; }
  if (result.won) { ui.showFeedback('You won!'); return; }
  if (result.done) { ui.showFeedback('You lost!'); return; }
  ui.showFeedback(`Try again. ${game.history.length} guesses so far.`);
});

```

Key decoupling moves

- `Game` exposes `history` publicly (via a copy) — no more reaching into UI to read it.
- `Game` takes `allowDuplicates` in its constructor — it doesn't consult the UI.
- `UI` exposes `getGuess()` and `showFeedback()` — doesn't know about game rules.

Gotchas & warnings

- **Forgetting the radix** in `parseInt`. Always pass `10`.
- **Logic leaking into UI.** If your UI object is computing game rules, it's no longer "just UI."

- **Reading UI state from inside the Game.** Pass it in via the constructor or method args — don't query the DOM from inside the model.

Mental model

Separate the **what** (Game — rules, state) from the **how** (UI — DOM, events). Wire them with event handlers. Each piece is independently testable.

Try it yourself

Refactor a small `while (true)` prompt-based game into three pieces: a `Game` class, a `ui` object, and an event handler that connects them. Test by swapping the UI (e.g. from prompt to DOM inputs) without touching the Game.

Episode 24 — Improving The User Experience

TL;DR

Replace `alert()` / `prompt()` with visible DOM feedback. Append guesses to a list, clear the input after each guess, re-focus the input, bundle resets into one `ui.reset()` method. UX matters — use your own app and get real user feedback.

PHP → JS differences

- **DOM feedback vs modal dialogs** — JS-heavy apps avoid modals for routine feedback

Concepts, explained

Appending history as DOM elements

```
const historyList = document.getElementById('historyList');
function addGuessToHistory(guess) {
  historyList.innerHTML += `<li>${guess}</li>`;
}
```

Resetting history in the DOM

```
function resetHistory() {
```

```
} historyList.innerHTML = '';
```

Clearing and re-focusing the input

```
const input = document.getElementById('guessInput');  
function resetGuess() {  
  input.value = '';  
  input.focus();  
}
```

Feedback in the DOM instead of alert

```
const feedback = document.getElementById('feedback');  
function showFeedback(msg) {  
  feedback.textContent = msg;  
}
```

Bundling resets

```
const ui = {  
  resetHistory() {  
    document.getElementById('historyList').innerHTML = '';  
  },  
  resetGuess() {  
    const input = document.getElementById('guessInput');  
    input.value = '';  
    input.focus();  
  },  
  showFeedback(msg) {  
    document.getElementById('feedback').textContent = msg;  
  },  
  reset() {  
    this.resetHistory();  
    this.resetGuess();  
    this.showFeedback('');  
  },  
};  
  
// After a game ends:  
ui.reset();
```

Use your own app

Developers think about their apps differently than users. After every meaningful change, use it as a user would. Get real user feedback — someone outside your head will spot UX issues you can't see.

Gotchas & warnings

- `innerHTML +=` **rebuilds the subtree**. Fine for small lists, slow for huge ones. Use `appendChild` with `createElement` for performance.
- `input.focus()` **on mobile** sometimes brings up the keyboard unexpectedly. Test on real devices.
- **Interpolating user input into `innerHTML`** is an XSS risk. Escape or use `textContent`.

Mental model

A good JS app is chatty in the best way — it reflects every action back to the user immediately. Bundle the common reset logic so calling it is one line.

Try it yourself

Take any alert/prompt-based mini-app and convert all its output to DOM elements. Add a `reset()` that clears everything and re-focuses the first input.

Episode 25 — Disabling Forms

TL;DR

You can't disable a whole form in one shot. Iterate `form.elements` and set `.disabled = true` on each. Three iteration styles: classic `for`, spread `[...form.elements].forEach`, or `Array.from(form.elements).forEach`. `NodeList`-like collections are *array-like*, not arrays — know the difference.

PHP → JS differences

- **Array-like vs real arrays** is a JS-specific pitfall — PHP doesn't distinguish

Concepts, explained

No blanket `form.disabled`

HTML has no "disable whole form" switch. You must disable each control.

Classic `for` — fastest

```
const form = document.getElementById('myForm');

function disableForm() {
  for (let i = 0; i < form.elements.length; i++) {
    form.elements[i].disabled = true;
  }
}

function enableForm() {
  for (let i = 0; i < form.elements.length; i++) {
    form.elements[i].disabled = false;
  }
}
```

Spread + `forEach`

```
[...form.elements].forEach(el => el.disabled = true);
```

`Array.from` + `forEach`

```
Array.from(form.elements).forEach(el => el.disabled = true);
```

Prototype-borrowing (older trick)

```
Array.prototype.forEach.call(form.elements, el => el.disabled = true);
```

Works because `form.elements` is array-like (has `.length` and indexed access) — `.call` lets you borrow `Array.prototype.forEach` even for non-array collections.

Array-like vs real array

```
const nodeList = document.querySelectorAll('li'); // NodeList — has .forEach
const formEls = form.elements;                 // HTMLFormControlsCollection — no .forEach

nodeList.forEach(...) // works
formEls.forEach(...)  // TypeError

[...formEls].forEach(...) // works (convert first)
```

Visual feedback with CSS

```
form.classList.add('opacity-50'); // Tailwind utility for disabled look
// ... later
form.classList.remove('opacity-50');
```

Gotchas & warnings

- `form.elements` **excludes divs and labels**. Only real form controls (inputs, buttons, selects, textareas). That's usually what you want.
- **Not every DOM collection has `.forEach`**. `querySelectorAll` does; `getElementsByName` doesn't. If in doubt, spread into a real array.
- **Classic `for` is fastest**. Rarely matters for 10 elements; matters for 10,000.

Mental model

Form controls are a collection you iterate. The collection might not be a real array, so either use classic `for`, spread it, or `Array.from` it. Disable each control one by one.

Try it yourself

Write `toggleForm(form, enabled)` that enables or disables every control in the form based on the boolean. Use classic `for` for speed.

Episode 26 — Using Callbacks

TL;DR

A callback is a function you hand to another object, to be called later. Store on `this.#onEnd`, check `typeof fn === 'function'` before calling. Single-callback APIs are limiting — support an array of callbacks with `addOnEnd(fn)`. This foreshadows custom events (Ep 27).

PHP → JS differences

- **Callbacks are first-class in JS** — PHP has `callable` but passing closures around is less fluent

Concepts, explained

Single-callback pattern

```
class Game {
  #onEnd;
  #history = [];
  #maxAttempts = 5;
  #secret = 7;

  set onEnd(fn) {
    this.#onEnd = fn;
  }

  checkGuess(guess) {
    this.#history.push(guess);

    const won = guess === this.#secret;
    const done = won || this.#history.length === this.#maxAttempts;

    if (done && typeof this.#onEnd === 'function') {
      this.#onEnd({ won, secret: this.#secret });
    }
  }
}

const game = new Game();
game.onEnd = (result) => {
  console.log(result.won ? 'You won!' : `You lost. It was ${result.secret}`);
};
```

Always check before calling

```
if (typeof this.#onEnd === 'function') {
  this.#onEnd(payload);
}
```

Without the check, assigning anything other than a function crashes when you try to invoke it.

Single-callback limitation

Only one listener. Assigning `game.onEnd = fnB` overwrites `fnA`.

Fix — an array of callbacks

```
class Game {
  #onEndCallbacks = [];

  addOnEnd(fn) {
```

```

    if (typeof fn === 'function') this.#onEndCallbacks.push(fn);
  }

  #fireOnEnd(payload) {
    this.#onEndCallbacks.forEach(fn => fn(payload));
  }

  checkGuess(guess) {
    // ... when game ends:
    this.#fireOnEnd({ won: true, secret: 7 });
  }
}

const game = new Game();
game.addOnEnd(result => console.log('A:', result));
game.addOnEnd(result => console.log('B:', result));

// Both fire on end

```

Foreshadowing

This is basically re-implementing the event system. Ep 27 replaces this with real custom events.

Gotchas & warnings

- **Forgetting `typeof === 'function'` checks.** Calling a non-function throws.
- **Overwriting with a single property.** Assigning `obj.onEnd = fn` overwrites any previous. `addOnEnd(fn)` is scale-safe.
- **Forgetting to pass payload to the callback.** If the callback wanted `result.won`, you'd better provide it.

Mental model

A callback is "tell me later." You hand a function to an object; the object calls it at the right moment with relevant data. Arrays of callbacks give you many listeners; custom events (next episode) are the cleaner long-term answer.

Try it yourself

Build a `Timer` class with `addOnTick(fn)` and `addOnDone(fn)`. Start a countdown from 5 to 0, calling all tick callbacks each second and all done callbacks when it reaches 0.

Episode 27 — Real Custom Events

TL;DR

Custom events decouple your code — anyone can listen without knowing about the emitter. Use `new Event('name')` for simple events, `new CustomEvent('name', { detail: {...} })` when you need to pass

data. Access via `event.detail.x`.

PHP → JS differences

- **Events in JS are the standard way to decouple** in both DOM and non-DOM contexts — PHP has events but they live in frameworks (Laravel Events, Symfony Events)

Concepts, explained

Basic custom event

```
// Dispatch
document.dispatchEvent(new Event('gameOver'));

// Listen
document.addEventListener('gameOver', () => {
  console.log('game is over');
});
```

Event with data — `CustomEvent` and `detail`

```
// Dispatch with payload
document.dispatchEvent(new CustomEvent('gameOver', {
  detail: { secret: 7, won: true },
}));

// Listen — access via event.detail
document.addEventListener('gameOver', (event) => {
  console.log(event.detail.secret, event.detail.won);
});
```

The `detail` key is the conventional location for custom event payloads.

Decoupled — anyone can listen

```
// In the game module
class Game {
  endGame(won) {
    document.dispatchEvent(new CustomEvent('gameOver', {
      detail: { won, secret: this.#secret },
    }));
  }
}

// In the UI module
document.addEventListener('gameOver', (event) => {
  ui.showFeedback(event.detail.won ? 'You won!' : `Lost. It was ${event.detail.secret}`);
});

// In the analytics module
document.addEventListener('gameOver', (event) => {
  analytics.track('game_finished', event.detail);
});
```

None of these know about each other.

Guess-specific event

```
document.dispatchEvent(new CustomEvent('gameGuess', {
  detail: { guess: 5, result: 'too low', remainingAttempts: 3 },
}));

document.addEventListener('gameGuess', (event) => {
```

```
const { guess, result, remainingAttempts } = event.detail;
ui.addHistoryItem(guess, result);
ui.showFeedback(`${result}. ${remainingAttempts} attempts left.`);
});
```

UI events too — even more decoupling

```
// UI dispatches "user submitted a guess"
document.dispatchEvent(new CustomEvent('uiSubmitGuess', {
  detail: { guess: input.value, game },
}));

// Handler wires UI to Game
document.addEventListener('uiSubmitGuess', (event) => {
  event.detail.game.checkGuess(event.detail.guess);
});
```

Gotchas & warnings

- **detail** is a specific key — not just anywhere on the event. `event.detail.x`, not `event.x`.
- Regular **Event** doesn't support **detail**. Use **CustomEvent** for data.
- Custom events don't bubble by default — covered in Ep 28.
- **Typos in event names** fail silently. Extract names as constants if they're used in multiple files.

Mental model

Events are an anonymous broadcast system. The emitter shouts "game over!" to a room; anyone listening for that shout reacts. Nobody needs references to each other.

Try it yourself

Build two modules: a **Cart** that dispatches `cartItemAdded` with `detail: { item, total }`, and a **CartBadge** that listens and updates a DOM counter. Prove they don't import each other.

Episode 28 — Blowing (Event) Bubbles

TL;DR

Built-in events bubble: they fire at the target then travel up through ancestors. This enables one document-level listener to handle many elements. Custom events do NOT bubble by default — opt in with `{ bubbles: true }`.

PHP → JS differences

- **Event bubbling is a DOM concept** — no PHP equivalent

Concepts, explained

Built-in events bubble

```
<div id="outer">
  <div id="middle">
    <button id="inner">Click</button>
  </div>
</div>
```

```
document.addEventListener('click', (event) => {
  console.log('document saw a click on', event.target);
});
```

Click the button — `document` sees it, because the event bubbles from the button up through `#middle`, `#outer`, `body`, `html`, and finally `document`.

Event delegation — one listener for many elements

```
document.getElementById('todoList').addEventListener('click', (event) => {
  if (event.target.classList.contains('delete-btn')) {
    event.target.closest('li').remove();
  }
});
```

No need to attach a listener to every delete button — one listener on the parent handles them all.

Custom events — opt in to bubbling

```
// Doesn't bubble by default
const el = document.getElementById('widget');
el.dispatchEvent(new CustomEvent('myEvent', { detail: { x: 1 } }));

document.addEventListener('myEvent', () => {
  // Never fires — event didn't bubble up
});

// Now it bubbles
el.dispatchEvent(new CustomEvent('myEvent', {
  bubbles: true,
  detail: { x: 1 },
}));

document.addEventListener('myEvent', () => {
  // Fires!
});
```

Regular `Event` also accepts `bubbles`

```
el.dispatchEvent(new Event('myEvent', { bubbles: true }));
```

But no `detail` — use `CustomEvent` for payloads.

Decision rule

- Dispatching **at `document`** → bubbling doesn't matter (already at the top)
- Dispatching **from an element**, listening at `document`/form/other ancestor → opt in to bubbling

Gotchas & warnings

- **Custom events not bubbling by default** is a common "why isn't my listener firing?" bug.
- **Some events don't bubble at all** (`focus` , `blur` , `load`). Use their bubbling counterparts (`focusin` , `focusout`) or listen on the element directly.
- **Stop propagation with caution.** `event.stopPropagation()` prevents parents from seeing an event, which can break other listeners you didn't write.

Mental model

Events start at the target and travel up. Listeners anywhere along the path can catch them. Custom events have this turned off unless you ask for it. For delegation (handling many elements with one listener), bubbling is essential.

Try it yourself

Build a `<ul>` with 5 `<li>`s. Attach a single click listener to the `<ul>` that logs which `<li>` was clicked. Use `event.target` to identify the one.

Episode 29 — Organize Your Apps With Modules

TL;DR

Split your code across files with ES Modules. Use `<script type="module">` to opt in. Each module has its own scope — no accidental globals, no IIFEs needed. Two export kinds: `export default` (one per file, imported by any name) and `export const/function` (named, imported by exact name).

PHP → JS differences

- **ES Modules are closer to PHP's namespace/require/use system** than old-school `<script>` tags were
- **Each module is its own scope** — similar to how PHP files generally don't leak variables

Concepts, explained

Plain `<script>` — shared global scope


```
<script src="game.js"></script>
<script src="ui.js"></script>
<!-- Both files share the global window scope -->
```

Any variable one file creates is visible to the other. Conflicts and pollution are easy.

Modules — isolated scope

```
<script type="module" src="main.js"></script>
```

Named exports

```
// ui.js
export const ui = {
  showFeedback(msg) { /* ... */ },
};

export function init() {
  console.log('UI ready');
}
```

```
// main.js
import { ui, init } from './ui.js';

ui.showFeedback('hi');
init();
```

Default export

```
// game.js
export default class Game {
  // ...
}
```

```
// main.js
import Game from './game.js'; // can be named anything
// also valid:
// import Hello from './game.js';

const g = new Game();
```

One default per file. The import name is arbitrary.

Mixing default and named

```
// game.js
export default class Game {}
export const VERSION = '1.0';
```

```
// main.js
import Game, { VERSION } from './game.js';
```

Renaming during import

```
import { ui as userInterface } from './ui.js';
```

Re-exports

```
// index.js
export { Game } from './game.js';
export { ui } from './ui.js';

// Elsewhere
import { Game, ui } from './index.js';
```

Gotchas & warnings

- **Modules require a web server or specific dev setup.** Opening `index.html` directly from the filesystem (`file://`) won't load modules — use a dev server (Vite, or even `python -m http.server`).
- **Paths need the extension.** `import Game from './game.js'` , not `./game` — browsers don't guess extensions.
- **Default export names are just local aliases.** Pick descriptive names to keep code readable.

Mental model

A module is an isolated bubble. Whatever you `export` , others can `import` . Whatever you don't export stays private. No more IIFEs — modules do it natively.

Try it yourself

Split an existing single-file app into `game.js` (default export: `Game` class), `ui.js` (named export: `ui` object), and `main.js` (imports both, wires them up). Load from `index.html` with `<script type="module">` .

Episode 30 — Serializing And Deserializing Objects

TL;DR

`JSON.stringify(obj)` converts an object to a JSON string; `JSON.parse(str)` does the reverse. Use for sending to servers, storing in browsers, or deep-cloning plain data. Can't serialize functions, undefined, or circular references.

PHP → JS differences

- `json_encode` / `json_decode` in PHP are direct equivalents
- **JSON requires double-quoted string keys** in both languages

Concepts, explained

Basic serialization

```
const person = { name: 'Ben', age: 23 };

const json = JSON.stringify(person);
console.log(json); // '{"name":"Ben","age":23}'

const obj = JSON.parse(json);
console.log(obj.name); // 'Ben'
```

Pretty-printing

```
JSON.stringify(person, null, 2);
/*
{
  "name": "Ben",
  "age": 23
}
*/
```

Deep clone trick

```
const original = { user: { name: 'Ben', roles: ['admin'] } };
const clone = JSON.parse(JSON.stringify(original));

clone.user.name = 'Other';
console.log(original.user.name); // 'Ben' — unchanged
```

(Modern alternative: `structuredClone(original)` — built-in since 2022.)

What can't be serialized

```
JSON.stringify({
  fn: function () {}, // dropped
  undef: undefined, // dropped
  sym: Symbol('x'), // dropped
  // circular: circular // throws TypeError
});

// Result: '{}'
```

PHP analogue

```
$person = ['name' => 'Ben', 'age' => 23];
$json = json_encode($person);
$obj = json_decode($json, true);
```

Gotchas & warnings

- **Functions disappear.** If your object has methods, they won't survive a round-trip.
- `undefined` is dropped (but `null` is kept).

- **Circular references throw.** `obj.self = obj` makes `JSON.stringify` crash.
- **Date objects become strings.** `new Date()` → ISO 8601 string on serialize. On parse you get a string, not a Date — rehydrate manually.

Mental model

JSON is a text format for plain data — strings, numbers, booleans, null, arrays, objects. Anything that is "just data" serializes fine. Anything with behaviour or identity (functions, symbols, circular refs) doesn't.

Try it yourself

Serialize a game state object (settings, history, score). Parse it back. Verify it round-trips. Now add a function to it and see what happens.

Episode 31 — Storing Stuff In Browsers

TL;DR

`localStorage` and `sessionStorage` share the same API. `localStorage` persists across tabs and sessions; `sessionStorage` is per-tab and dies when the tab closes. Values must be strings — pair with `JSON.stringify` / `JSON.parse` for objects.

PHP → JS differences

- **Session concept is different** — PHP sessions live on the server; JS Web Storage lives in the browser

Concepts, explained

The API (same for both)

```
localStorage.setItem('key', 'value'); // write
const v = localStorage.getItem('key'); // read — returns string or null
localStorage.removeItem('key'); // delete one
localStorage.clear(); // delete everything in this origin
```

Storing objects — serialize first

```
const state = { score: 42, history: [1, 2, 3] };
```

```
localStorage.setItem('gameState', JSON.stringify(state));
const loaded = JSON.parse(localStorage.getItem('gameState'));
console.log(loaded.score); // 42
```

localStorage vs sessionStorage

```
// sessionStorage — gone when this tab closes
sessionStorage.setItem('draft', 'hello');

// localStorage — persists forever (until user clears browser data)
localStorage.setItem('theme', 'dark');
```

Other tabs on the same origin see `localStorage` but not each other's `sessionStorage`.

Using a key constant

Avoids typos.

```
const STORAGE_KEY = 'gameState';

function save(state) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
}

function load() {
  const raw = localStorage.getItem(STORAGE_KEY);
  return raw ? JSON.parse(raw) : null;
}
```

Private fields won't serialize

`JSON.stringify` only sees enumerable properties — not `#privateFields` and not getters.

```
class Game {
  #secret = 7;
  history = [1, 2, 3];
}

const g = new Game();
JSON.stringify(g); // '{"history":[1,2,3]}' — #secret missing
```

Fix: build an explicit plain-state object.

```
class Game {
  #secret = 7;
  history = [];

  toStorage() {
    return { secret: this.#secret, history: [...this.history] };
  }

  static fromStorage(data) {
    const g = new Game();
    g.#secret = data.secret;
    g.history = [...data.history];
    return g;
  }
}
```

Gotchas & warnings

- **Storage values are always strings.** `setItem('n', 5)` stores `"5"`. Use JSON for anything non-string.
- **Storage is synchronous** and lives on the main thread. Don't write huge blobs on every keystroke.

- **Storage has size limits** (~5-10 MB per origin depending on browser).
- `setItem` **overwrites**. No append — read, modify, write.

Mental model

Two key/value stores with identical APIs but different lifetimes. `session` = per-tab, throwaway. `local` = per-origin, persistent. Always pair with JSON for object data, and build explicit "what to store" shapes when classes have private state.

Try it yourself

Build a "save game" feature: serialize state on each move into `localStorage`; on page load, if saved state exists, restore it. Use a key constant.

Episode 32 — Promise Me

TL;DR

A Promise is a placeholder for a future value, with three states: pending, fulfilled, rejected. Create with `new Promise((resolve, reject) => { ... })`, consume with `.then()` / `.catch()`. Wrap sync operations in promises early if the underlying operation might go async later.

PHP → JS differences

- **Promises are a JS concept** — PHP is historically synchronous, though modern PHP has async libraries (ReactPHP, Amp)

Concepts, explained

The restaurant analogy

You order food. The waiter hands you a pager (the Promise). You can do other things while waiting. Eventually the pager buzzes — your order is ready (fulfilled) or they're out (rejected).

Creating a promise

```
const promise = new Promise((resolve, reject) => {
  try {
    const result = 'success';
    resolve(result);
  } catch (err) {
    reject(err);
  }
});
```

```
});
```

Consuming a promise

```
promise
  .then(value => {
    console.log('got:', value);
  })
  .catch(err => {
    console.log('oops:', err);
  });
```

.then and **.catch** return promises — chainable

```
fetchUser()
  .then(user => fetchPosts(user.id))
  .then(posts => console.log(posts))
  .catch(err => console.error(err));
```

Each **.then** callback can return a value (wrapped in a new promise) or another promise (chained).

Why wrap sync operations

```
// Synchronous — can't later swap for async
function loadState() {
  return JSON.parse(localStorage.getItem('state'));
}

// Promise-wrapped — consistent API, swappable later
function loadState() {
  return new Promise((resolve, reject) => {
    try {
      const state = JSON.parse(localStorage.getItem('state'));
      resolve(state);
    } catch (err) {
      reject(err);
    }
  });
}

// Today: local
loadState().then(render);

// Tomorrow: swap to server
function loadState() {
  return fetch('/api/state').then(r => r.json());
}
loadState().then(render); // same call site
```

"Async is contagious"

Once one piece is async, everything depending on it becomes async. Plan async boundaries early.

Gotchas & warnings

- **Forgetting **.catch**** . Rejected promises without a catch → "Uncaught (in promise)" error in console.
- **Returning a value vs returning a promise inside **.then**** . Both work — JS wraps plain values automatically.
- **Chaining mistakes** . Forgetting to **return** inside a **.then** breaks the chain — the next **.then** gets **undefined** .

Mental model

A promise is "I'll have a value later." You attach handlers with `.then` (success) and `.catch` (failure). Once one operation in your pipeline is async, everything downstream becomes promise-shaped.

Try it yourself

Wrap `localStorage.getItem('state')` in a promise. Then swap the implementation to `fetch('/api/state')`. Verify the call site doesn't need to change.

Episode 33 — Wait For It

TL;DR

`async` marks a function as returning a Promise. `await` pauses inside an async function until a promise resolves, and gives you the resolved value. Turns `.then().then().catch()` chains into straight-line code.

PHP → JS differences

- **Async/await in JS is built-in** and widely used
- **PHP 8.1 has fibers** — similar idea, different syntax

Concepts, explained

Basic async/await

```
async function getUser() {
  const response = await fetch('/api/user');
  const data = await response.json();
  return data;
}

getUser().then(user => console.log(user));
```

Any `async` function returns a Promise implicitly.

Error handling with try/catch

```
async function getUser() {
  try {
    const response = await fetch('/api/user');
    if (!response.ok) throw new Error('Not found');
    return await response.json();
  } catch (err) {
    console.error('Failed:', err);
  }
}
```



```
    return null;
  }
}
```

Compare — with vs without async/await

Without (promise chain):

```
function loadGame() {
  return fetch('/api/game')
    .then(r => r.json())
    .then(game => {
      return fetch(`/api/users/${game.userId}`)
        .then(r => r.json())
        .then(user => ({ game, user }));
    })
    .catch(err => console.error(err));
}
```

With:

```
async function loadGame() {
  try {
    const game = await fetch('/api/game').then(r => r.json());
    const user = await fetch(`/api/users/${game.userId}`).then(r => r.json());
    return { game, user };
  } catch (err) {
    console.error(err);
  }
}
```

await only works inside **async**

```
// SyntaxError outside async
const data = await fetch('/api');

// OK
async function load() {
  const data = await fetch('/api');
}
```

(Top-level **await** is allowed in ES modules.)

Async propagates

If a function calls an async function and needs its result, the caller usually also becomes async.

```
async function getJson(url) {
  const r = await fetch(url);
  return r.json();
}

async function renderUser() {
  const user = await getJson('/api/user'); // requires async
  document.getElementById('name').textContent = user.name;
}
```

Gotchas & warnings

- **await** outside **async** is a **SyntaxError** (except top-level in ESM).
- **Forgetting await**. `const data = fetch(url)` gives you a Promise, not the data. Forgetting **await** is a classic beginner bug.
- **Parallel vs sequential**. Two awaits in a row run sequentially. Use `Promise.all([a, b])` if they

don't depend on each other.

Mental model

`await` is a pause button that waits for a promise, then gives you its value. It reads top-to-bottom like regular code, but under the hood each `await` is `.then()`. `async` marks "this function may pause; its return value is always a promise."

Try it yourself

Rewrite a promise-chain version of an API call using `async/await`. Add a `try/catch` for errors. Verify the behaviour matches.

Episode 34 — Fetch Me That Data!

TL;DR

`fetch(url)` returns a Promise of a `Response`. Check `response.ok` for HTTP success. Call `.json()`, `.text()`, or `.blob()` (each returns a Promise) to read the body. Use `URLSearchParams` to build query strings. Inspect requests in DevTools → Network.

PHP → JS differences

- **`fetch` is JS's async HTTP client** — PHP has `curl`, `Guzzle`, or `file_get_contents`
- **Two-step body reading** — first `await` the response, then `await` the body parsing

Concepts, explained

Basic GET

```
async function getUser() {
  const response = await fetch('/api/user');

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  const data = await response.json();
  return data;
}
```

`response.ok`

`true` if the HTTP status is in the 200-299 range. Note: `fetch` only rejects on network errors, NOT on 4xx/5xx responses — you have to check `ok` yourself.

Reading the body — different formats

```
response.text(); // Promise<string>
response.json(); // Promise<parsed object>
response.blob(); // Promise<Blob> – for binary data
```

Each returns a Promise — you must `await` or `.then()` them.

Query strings with `URLSearchParams`

```
const params = new URLSearchParams({
  q: 'javascript',
  page: 2,
  limit: 10,
});

const url = `/api/search?${params}`;
// /api/search?q=javascript&page=2&limit=10

const response = await fetch(url);
```

Custom headers

```
const response = await fetch('/api/protected', {
  headers: {
    Accept: 'application/json',
    Authorization: 'Bearer abc123',
  },
});
```

Throwing semantic errors

```
if (!response.ok) {
  throw new Error("response wasn't okay"); // better than throw {}
}
```

Errors have `.stack`, `.message`, `.name` — useful for debugging.

Inspecting in DevTools

Network tab → request → Headers (what you sent), Preview/Response (what came back), Timing (how long each phase took). Indispensable for debugging API issues.

Gotchas & warnings

- `fetch` doesn't reject on 4xx/5xx. Always check `response.ok`.
- **Two awaits required.** `await fetch(...)` gives you the response; you still need `await response.json()` to read the body.
- **Body can only be read once** per response. If you need to read twice, `response.clone()` first.
- **CORS errors** happen for cross-origin requests. The server must send the right headers.

Mental model

`fetch` is "send an HTTP request, give me a Promise of the Response." The response has metadata (status, headers) and a body that you parse separately via `.text()` / `.json()` / `.blob()`. Always check `response.ok` — fetch only errors on network issues.

Try it yourself

Build `getJSON(url)` — it fetches, checks `response.ok`, parses JSON, and throws on failure. Use it to load `https://jsonplaceholder.typicode.com/users/1` and log the user's name.

Episode 35 — Post It

TL;DR

Pass an options object to `fetch` to POST/PUT/PATCH/DELETE. Two common body types: `FormData` (multipart, browser sets Content-Type automatically) and JSON (must set `Content-Type: application/json` yourself). Default method is GET — no need to specify.

PHP → JS differences

- **Submitting forms as JSON** is a modern-web pattern — traditional PHP form POSTs were always multipart or urlencoded

Concepts, explained

POST with `FormData`

```
async function submitForm() {
  const fd = new FormData();
  fd.append('name', 'Ben');
  fd.append('email', 'ben@example.com');
  fd.append('history[]', 1);
  fd.append('history[]', 2);

  const response = await fetch('/api/signup', {
    method: 'POST',
    body: fd,
    // No Content-Type header — browser sets 'multipart/form-data' with boundary
  });

  return response.json();
}
```

All `FormData` values become strings on the server; you'll need to reconvert numbers/booleans.

POST with JSON

```

async function saveState(state) {
  const response = await fetch('/api/state', {
    method: 'POST',
    body: JSON.stringify(state),
    headers: {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
    },
  });
  if (!response.ok) throw new Error('Save failed');
  return response.json();
}

```

With JSON you **must** set `Content-Type` yourself — the browser won't.

Other methods

```

await fetch('/api/user/1', { method: 'DELETE' });
await fetch('/api/user/1', {
  method: 'PATCH',
  body: JSON.stringify({ name: 'Ben' }),
  headers: { 'Content-Type': 'application/json' },
});

```

Default method is GET

```

fetch('/api/user'); // GET — no options needed
fetch('/api/user', { method: 'GET' }); // same thing, explicit

```

Reusable helpers

```

async function getJson(url) {
  const response = await fetch(url, {
    headers: { Accept: 'application/json' },
  });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}

async function postJson(url, data) {
  const response = await fetch(url, {
    method: 'POST',
    body: JSON.stringify(data),
    headers: {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
    },
  });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}

// Usage
const user = await getJson('/api/user/1');
const saved = await postJson('/api/state', { score: 42 });

```

Gotchas & warnings

- **Must set Content-Type for JSON.** The server won't know to parse JSON otherwise.
- **Don't set Content-Type for FormData.** The browser generates the `multipart/form-data; boundary=...` header — overriding it breaks parsing.
- **FormData stringifies everything.** A number `5` becomes `"5"` on the server.
- **body must be a string, FormData, or a few other special types.** You can't pass a plain object directly — `JSON.stringify` it.

Mental model

`fetch(url, options)` — options controls method, body, headers. Default is GET without a body. For JSON bodies, serialize and set headers. For form submissions, `FormData` does everything. Wrap common patterns in helper functions.

Try it yourself

Build `postJson(url, data)` and use it to POST `{ name, email }` to `https://jsonplaceholder.typicode.com/users`. Log the response.

Course Synthesis — Recurring Themes

Six themes run through the whole course. Each is reinforced across multiple episodes — if these click, JS clicks.

1. Declare everything, avoid globals

Accidental globals are the #1 beginner bug. `let`, `const`, and modules kill them off. **Reinforced in:** Ep 1 (accidental globals), Ep 5 (IIFEs), Ep 6 (let/const/var), Ep 29 (modules).

2. Prefer `===` and use type-aware operators carefully

JS's coercion will bite you if you let it. Always use `===`. Understand truthy/falsy. **Reinforced in:** Ep 3 (operators), Ep 10 (`??` vs `||`), Ep 20 (`.checked` vs `.value`).

3. Functions are values — pass them, return them, store them

First-class functions unlock callbacks, events, and a lot of idiom. **Reinforced in:** Ep 5 (creating functions), Ep 11 (factories), Ep 13 (context/this), Ep 26 (callbacks), Ep 27 (events).

4. Options objects + destructuring beat positional args

For any function with more than ~2 configurable parameters, use an options object. **Reinforced in:** Ep 9 (options pattern), Ep 10 (destructuring), Ep 12 (static with options).

5. Events and listeners for decoupling

UI emits events; other modules listen. Nobody imports each other. **Reinforced in:** Ep 16 (event basics), Ep 19 (delegation), Ep 23 (event-driven refactor), Ep 27 (custom events), Ep 28 (bubbling).

6. Async is contagious — plan the boundaries

Once one thing awaits, everything downstream does too. Wrap sync operations in promises early if they might go async later. **Reinforced in:** Ep 32 (promises), Ep 33 (async/await), Ep 34 (fetch), Ep 35 (post).

Closing thought

The sharpest mental model: **JS looks like PHP but thinks like Scheme**. The syntax is C-family, but the underlying model — functions everywhere, objects as dictionaries, events and promises for control flow — is much closer to Lisp/Scheme than to PHP's imperative-OOP core. When you hit something confusing, ask "what would a language designer who loved functions do here?" and the answer is usually right.